



Teachers

Simone Viani

Software Architect R&D

email: simone.viani@sinfo-one.it

Luca Zaccomer

Analyst Developer R&D

email: luca.zaccomer@sinfo-one.it



Containerization

Lightweight resource virtualization / isolation
Same hardware and kernel of the Operating System

Original project from [Docker](#)
REST Server

- Engine as service daemon
- Execute client requests

Command Line Interface Client

- Set of unix-like sub-commands, easy mnemonics
- Build images, run containers, show logs, ...

Docker Containers

Linux Kernel features

- cgroups: limit resources usage, CPU, memory
- namespaces: resource visibility / partitioning

Filesystem features

- chroot: process apparent root directory
- union filesystems
 - OverlayFS

Network features

- Subnets for containers, local or across cluster
- NAT to map external:internal ports

Image and Registry

Registry

- Collection of published images
- Public or private

Image

- Built from sources
- Layers of base images
- Persistent on localhost
- Packaged filesystem in OCI format
- Immutable, SHA signed
- Distributable thru registries
- Executable as container

Container

Running instance of an image

Create a new volatile filesystem layer on top of the image

Group of isolated resources

- Processes: host PID mapping
- Networks: consume an IP on each subnet
- Filesystem: volatile
- Volumes: persistent, shareable

Stopped containers keep the volatile filesystem

Removed containers only keep volumes

Volumes

Mount points for running containers

Bind mount

- Map host paths into container paths

Volume mount

- Drivers for persistent filesystems
- Initialize content from the image on first mount

Docker Compose

CLI to manage groups of services

- as a whole
- as a subset / single

Defines

- Services
 - Run as containers
- Networks
 - Shared across the compose or external
- Volumes
 - Shared across the compose or external

Service Scaling

Service replica

- More than one container
- Dynamically scale up/down
- Dynamic IP:port of each instance
- Implicit balancer: round-robin of incoming requests

Explicit Load Balancer

- Static entry-point for the service
- Sidecar container, ie: [Envoy Proxy](#)



Load Balancer

Network / Transport layer (OSI L4)

- Connection oriented
- Host:port

Application layer (OSI L7)

- Message oriented
- HTTP URL, headers
- Sticky session
 - By HTTP header, ie: cookie



Health Check

Command to periodically self-check working status

- Reacheable server socket
- Files availability
- Request fulfillment
- Resources consumption, react to leaks

Restart on multiple consecutive failures

- Avoid false negative



Orchestration

DevOps

Practices/tools for improving systems development life cycle.

CI: Continuous Integration

CD: Continuous Delivery

Clusters

- [Docker Swarm](#)
- [Kubernetes](#)

